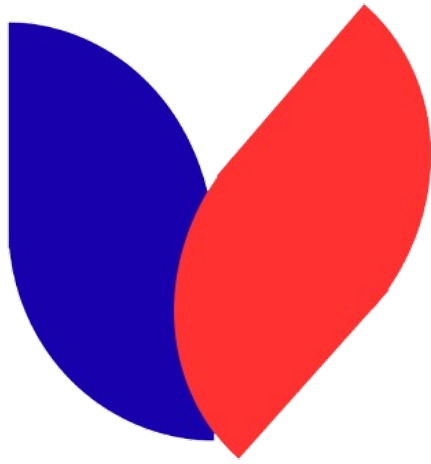




Learn Lopo

Lopo Programming Language Developer Guide (.lopo)

Lopo Core Team

June 26, 2026

Contents

1	Introduction to Lopo	5
1.1	Key Features of Lopo	5
1.2	Example Lopo Code	5
1.3	Philosophy of Lopo	5
2	Installation and Setup	5
2.1	Download Lopo	6
2.2	Installation (Windows)	6
2.3	Add Lopo to PATH	6
2.4	Verify Installation	6
2.5	Running a Program	7
3	Variables and Data Types	7
3.1	Defining Variables	7
3.2	Supported Data Types	7
3.3	Examples of Data Types	7
4	Operators	8
4.1	Arithmetic Operators	8
4.2	Comparison Operators	8
4.3	Logical Behavior Note	9
5	Control Flow	9
5.1	If / Else	9
6	Loops	10
6.1	For-In Loop	10
6.2	For-Let In Loop	10
6.3	While Loop	10
7	Functions	11
7.1	Function Declaration	11
7.2	Function Expression	11
7.3	Function Behavior	11
7.4	Key Features	11

8	Entities and Objects	11
8.1	Defining an Entity	12
8.2	Entity Evaluation Model	12
8.3	Creating an Instance	12
8.4	Self Keyword	13
8.5	Inheritance System	13
8.6	Inheritance Evaluation Model	13
8.7	Runtime Structure of an Entity	13
8.8	Design Summary	14
9	Error Handling	14
10	Built-in Commands	14
10.1	Input and Output	14
10.2	String Utilities	14
10.3	Type Utilities	15
10.4	Array Utilities	15
10.5	Object Utilities	15
10.6	Math Utilities	16
10.7	String Transformation Utilities	17
10.8	File System Utilities	17
10.9	System Utilities	17
10.10	Network Utilities	17
10.11	Encoding Utilities	18
10.12	Data Structures Utilities	18
10.13	Utility Summary	18
11	File Import System	18
12	Advanced Concepts	18
13	Appendix A: Syntax Quick Reference	19
13.1	Variables	19
13.2	Data Types	19
13.3	Output	19
13.4	Input	19
13.5	Operators	19
13.6	If / Else	19
13.7	Loops	20
13.7.1	For-In Loop	20
13.7.2	For-Let In Loop	20
13.7.3	While Loop	20
13.8	Functions	20
13.9	Entities	20
13.10	Inheritance	20
13.11	Arrays	21
13.12	Objects	21
13.13	Built-in Commands	21

13.14	Built-in Functions	21
13.15	File Operations	22
13.16	Math Utilities	22
13.17	String Utilities	22
13.18	Arrays Utilities	23
13.19	Object Utilities	23
14	Appendix B: Example Programs	23
14.1	Simple Calculator	23
14.2	Loop with Condition Filtering	24
14.3	Object Processing Example	24
14.4	Entity Usage Example	24
14.5	String Manipulation Example	24
14.6	Array Transformation Example	25
14.7	File Operations Example	25
14.8	Random Utility Example	25
14.9	Recursive Function Example	25

1 Introduction to Lopo

Lopo is a lightweight interpreted programming language designed to prioritize simplicity, readability, and extensibility. It is built with the goal of making programming more accessible while still providing powerful features for advanced users.

Unlike traditional programming languages that often rely on complex syntax, Lopo uses a clean and structured style with keywords such as **define**, **show**, and **for in**. This allows developers to focus more on logic and problem-solving rather than syntax complexity.

Lopo is executed through its custom interpreter, which reads and evaluates `.lopo` files line by line using an abstract syntax tree (AST) execution model. This design enables flexible runtime behavior, dynamic evaluation, and extensibility.

1.1 Key Features of Lopo

- Simple and human-readable syntax
- Interpreted execution model (AST-based)
- Dynamic typing system
- Built-in control flow: if, else, while, for-in
- Function and entity-based architecture
- Flexible object and array handling

1.2 Example Lopo Code

```
define message = "Hello from Lopo";
show message;

define add = func(a, b) {
    return a + b;
};

show add(5, 10);
```

This example demonstrates variable declaration, function creation, and output using the **show** command.

1.3 Philosophy of Lopo

The philosophy behind Lopo is to reduce unnecessary complexity in programming languages while maintaining enough power to build real-world applications. It focuses on readability, experimentation, and developer-friendly design.

2 Installation and Setup

Lopo is distributed as a pre-built CLI executable. The easiest way to install Lopo is by downloading the latest stable release.

2.1 Download Lopo

Go to the official Lopo documentation repository:

<https://github.com/dominexmacedon-docs/lopo-docs>

Navigate to the **Releases** section and download the latest version of Lopo for your operating system.

2.2 Installation (Windows)

After downloading the release:

1. Create a folder at:

`C:\Lopo`

2. Move the downloaded `lopo.exe` file into this folder.

3. Rename the file to:

`lopo.exe`

2.3 Add Lopo to PATH

To run Lopo from anywhere in the terminal:

1. Press **Windows + S** and search for:

Environment Variables

2. Open **Edit the system environment variables**.
3. Click **Environment Variables**.
4. Under **System variables**, find and select **Path**, then click **Edit**.
5. Click **New** and add:

`C:\Lopo`

6. Click **OK** on all windows to save changes.

2.4 Verify Installation

Open a new Command Prompt and run:

`lopo --version`

If installed correctly, you will see the installed Lopo version printed.

2.5 Running a Program

To run a Lopo program, use:

```
lopo filename.lopo
```

This executes the file using the Lopo.

3 Variables and Data Types

In Lopo, variables are created using the **define** keyword. The language is dynamically typed, meaning the type of a variable is determined automatically at runtime based on the assigned value.

Variables in Lopo can store different types of data, including strings, numbers, booleans, arrays, and objects. This flexibility allows developers to write concise and expressive code without explicit type declarations.

3.1 Defining Variables

```
define message = "Hello Lopo";  
define count = 10;  
define isActive = true;
```

3.2 Supported Data Types

Lopo supports the following core data types:

- **String** — Text values enclosed in quotes
- **Number** — Integer and decimal values
- **Boolean** — true or false values
- **Array** — Ordered list of values
- **Object** — Key-value structured data

3.3 Examples of Data Types

```
define name = "Starlight";  
define age = 25;  
define isDev = true;  
  
define arr = [1, 2, 3, 4];  
  
define obj = {  
    name: "Lopo",  
    version: 1  
};
```

Variables in Lopo are mutable, meaning their values can be changed at any time during execution using assignment expressions.

4 Operators

Lopo provides a set of operators that allow developers to perform calculations, comparisons, and logical evaluations. These operators are designed to be simple and consistent with the language's readable syntax style.

4.1 Arithmetic Operators

Arithmetic operators are used to perform mathematical operations on numbers.

- + Addition
- - Subtraction
- * Multiplication
- / Division
- % Modulus (remainder)

```
define a = 10;
define b = 3;

show a + b;
show a - b;
show a * b;
show a / b;
show a % b;
```

4.2 Comparison Operators

Comparison operators are used to compare two values and return a boolean result.

- == Equal to
- != Not equal to
- < Less than
- <= Less than or equal to
- > Greater than
- >= Greater than or equal to

```
define x = 5;
define y = 10;

show x == y;
show x != y;
show x < y;
show x <= y;
show x > y;
show x >= y;
```

4.3 Logical Behavior Note

Lopo does not use explicit logical keywords such as **AND** or **OR**. Instead, logical behavior is handled through expression evaluation rules inside conditions.

For example:

```
define a = 10;
define b = 20;

if (a < b) {
    show "Condition passed";
}
```

In Lopo, any expression that evaluates to a truthy value is considered true in control flow statements.

5 Control Flow

Control flow in Lopo allows programs to make decisions and execute different blocks of code based on conditions. The primary control structure is the **if / else** statement.

5.1 If / Else

The **if** statement is used to execute a block of code only when a given condition evaluates to true. If the condition is false, an optional **else** block can be executed instead.

```
define x = 10;

if (x > 5) {
    show "x is greater than 5";
} else {
    show "x is 5 or less";
}
```

Conditions in Lopo are evaluated dynamically at runtime. Any expression that results in a truthy value will execute the **if** block, while falsy values will trigger the **else** block.

Multiple conditions can also be chained using nested **if** statements:

```
define score = 85;

if (score >= 90) {
    show "Grade A";
} else {
    if (score >= 75) {
        show "Grade B";
    } else {
        show "Grade C";
    }
}
```

This structure allows developers to build clear decision-making logic in their Lopo programs.

6 Loops

Loops in Lopo are used to execute a block of code repeatedly. The language supports three main loop types: `for in`, `for let in`, and `while` loops. These provide flexibility for iterating over data structures and controlling repeated execution.

6.1 For-In Loop

The `for in` loop is used to iterate over arrays and objects. When used with arrays, it iterates over values. When used with objects, it iterates over keys.

```
define arr = [10, 20, 30];

for x in arr {
    show x;
}
```

For objects:

```
define obj = {
    a: 1,
    b: 2,
    c: 3
};

for key in obj {
    show key;
}
```

6.2 For-Let In Loop

The `for let in` loop is similar to `for in`, but it creates a new block scope for each iteration. This ensures that variables defined inside the loop do not leak into the outer scope.

```
for let x in [10, 20, 30, 40] {
    show x;
}
```

This is useful when working with temporary loop variables that should not affect outer variables.

6.3 While Loop

The `while` loop repeats a block of code as long as a condition remains true. The condition is evaluated before each iteration.

```
define i = 0;

while (i < 5) {
    show i;
    i = i + 1;
}
```

The loop will continue until the condition becomes false. Care must be taken to avoid infinite loops by properly updating loop variables inside the body.

7 Functions

Functions in Lopo are used to group reusable blocks of code into a single unit. They allow developers to write modular, clean, and maintainable programs. Lopo supports both function declarations and function expressions.

7.1 Function Declaration

A function declaration defines a named function using the **func** keyword. These functions are stored in the environment and can be called anywhere after definition.

```
define add = func(a, b) {  
    return a + b;  
};  
  
show add(5, 10);
```

7.2 Function Expression

Function expressions allow functions to be assigned to variables. These functions can be anonymous or named depending on usage.

```
define multiply = func(x, y) {  
    return x * y;  
};  
  
show multiply(3, 4);
```

7.3 Function Behavior

Functions in Lopo create a new execution environment when called. Parameters are passed by value into this environment, and the function returns a result using the **return** statement.

7.4 Key Features

- First-class functions (can be stored in variables)
- Support for recursion
- Lexical scoping using environments
- Return-based output handling

8 Entities and Objects

Lopo provides an entity system that allows developers to define structured objects with methods, state, and optional inheritance. Entities are runtime objects constructed and stored in the environment, supporting object-oriented programming patterns.

8.1 Defining an Entity

Entities are declared using the `entity` keyword. Each entity contains methods and optional initialization logic.

```
entity Person {
  init(name, age) {
    self.name = name;
    self.age = age;
  }

  greet() {
    show "Hello " + self.name;
  }
}
```

8.2 Entity Evaluation Model

At runtime, entities are evaluated using the internal evaluator function:

- EntityDeclaration is handled by `evalEntity`
- Methods are stored in an internal method table
- Each entity keeps a reference to its environment
- The entity is registered in the current environment

Internally, an entity is represented as:

```
{
  name: "Person",
  methods: { ... },
  parent: null,
  env: Environment
}
```

8.3 Creating an Instance

Entities are instantiated using the `new` keyword. This triggers `evalEntityNew`, which creates a runtime object from the entity definition.

```
define p = new Person("Alex", 25);
```

```
p.greet();
```

Each instance:

- Creates a new object scope
- Injects `self` into method execution context
- Binds methods dynamically from the entity definition

8.4 Self Keyword

The **self** keyword refers to the current instance inside an entity method.

Internally, **self** is stored in the environment:

```
case 'SelfExpression':
    return env.get('self', node, this.source);
```

8.5 Inheritance System

Lopo supports single inheritance using the **inherits** keyword.

```
entity Animal {
    speak() {
        show "Some sound";
    }
}

entity Dog inherits Animal {
    speak() {
        show "Bark";
    }
}
```

8.6 Inheritance Evaluation Model

Inheritance is resolved at runtime using **evalInherits**:

- Parent entity is evaluated from environment
- Parent must contain a valid method table
- Child entity references parent for method lookup

Internally:

```
if (node.inherits) {
    entity.parent = await this.evaluate(node.inherits, env);
}
```

8.7 Runtime Structure of an Entity

An entity in Lopo runtime looks like:

```
{
  name: "Dog",
  methods: {
    speak: function,
    init: function
  },
  parent: Animal,
  env: Environment
}
```

8.8 Design Summary

The entity system in Lopo is designed to be lightweight and flexible. It avoids heavy class-based compilation and instead relies on runtime evaluation, environment chaining, and dynamic method resolution.

This makes entities:

- Fast to interpret
- Easy to extend
- Fully dynamic at runtime

9 Error Handling

try/catch equivalent using runtime signals.

10 Built-in Commands

Lopo provides a rich standard library of built-in functions that simplify common programming tasks such as string manipulation, array processing, mathematics, file handling, and utility operations.

These built-ins are globally available and can be used without importing any module.

10.1 Input and Output

- `show(value)` — Prints output to the console
- `ask(prompt)` — Reads user input from the console

10.2 String Utilities

Lopo includes powerful string manipulation tools:

- `len(value)` — Returns length of string, array, or object
- `lower(str)` — Converts string to lowercase
- `upper(str)` — Converts string to uppercase
- `trim(str)` — Removes whitespace from both ends
- `toString(value)` — Converts any value to string
- `startsWith(str, prefix)` — Checks prefix match
- `endsWith(str, suffix)` — Checks suffix match
- `includes(str, substr)` — Checks substring existence
- `replace(str, search, replacement)` — Replaces text
- `repeat(str, times)` — Repeats a string multiple times

10.3 Type Utilities

- `type(value)` — Returns data type
- `typeof(value)` — Enhanced type detection (includes null and array)
- `isNaN(value)` — Checks if value is not a number
- `num(value)` — Converts value to number safely
- `str(value)` — Converts value to string

10.4 Array Utilities

- `map(array, fn)` — Transforms each element
- `filter(array, fn)` — Filters elements based on condition
- `reduce(array, fn, initial)` — Reduces array to single value
- `push(arr, value)` — Adds element to array
- `pop(arr)` — Removes last element
- `shift(arr)` — Removes first element
- `unshift(arr, value)` — Adds element at start
- `sort(arr)` — Sorts array
- `reverse(arr)` — Reverses array
- `unique(arr)` — Removes duplicates
- `flatten(arr)` — Flattens nested arrays
- `indexOf(arr, value)` — Finds index of element
- `includesArr(arr, value)` — Checks existence
- `randomChoice(arr)` — Returns random element
- `count(arr, value)` — Counts occurrences
- `uniqueBy(arr, fn)` — Unique by computed key

10.5 Object Utilities

- `keys(obj)` — Returns object keys
- `values(obj)` — Returns object values
- `entries(obj)` — Returns key-value pairs
- `has(obj, key)` — Checks key existence
- `hasOwn(obj, key)` — Safe property check

- `merge(obj1, obj2)` — Shallow merge
- `mergeDeep(obj1, obj2)` — Deep merge
- `invert(obj)` — Swaps keys and values
- `isEmpty(value)` — Checks emptiness
- `clone(value)` — Shallow copy
- `deepClone(value)` — Deep copy
- `getProp(obj, path)` — Safe nested access
- `setProp(obj, path, value)` — Safe nested assignment

10.6 Math Utilities

- `random(min, max)` — Random integer
- `randomFloat(min, max)` — Random float
- `randomInt(min, max)` — Inclusive random integer
- `floor(x)` — Floor value
- `ceil(x)` — Ceiling value
- `round(x)` — Round value
- `abs(x)` — Absolute value
- `pow(a, b)` — Power function
- `sqrt(x)` — Square root
- `min(...args)` — Minimum value
- `max(...args)` — Maximum value
- `clamp(value, min, max)` — Limits value range
- `sign(value)` — Returns sign
- `lerp(a, b, t)` — Linear interpolation
- `degToRad(deg)` — Degrees to radians
- `radToDeg(rad)` — Radians to degrees

10.7 String Transformation Utilities

- `capitalize(str)` — Capitalizes first letter
- `reverseStr(str)` — Reverses string
- `trimStart(str)` — Left trim
- `trimEnd(str)` — Right trim
- `camelCase(str)` — Converts to camelCase
- `kebabCase(str)` — Converts to kebab-case
- `repeatStr(str, n)` — Repeats string

10.8 File System Utilities

- `readFile(path)` — Reads file content
- `writeFile(path, content)` — Writes file
- `appendFile(path, content)` — Appends to file
- `deleteFile(path)` — Deletes file
- `exists(path)` — Checks file existence
- `mkdir(path)` — Creates directory
- `rmdir(path)` — Removes directory
- `readJSON(path)` — Reads JSON file
- `writeJSON(path, obj)` — Writes JSON file

10.9 System Utilities

- `timestamp()` — Current timestamp
- `now()` — Current time
- `uuid()` — Generates unique ID
- `sleep(ms)` — Async delay

10.10 Network Utilities

- `fetch(url)` — HTTP request
- `get(url)` — GET JSON request
- `post(url, data)` — POST JSON request

10.11 Encoding Utilities

- `encodeURIComponent(value)` — URL encoding
- `JSONParse(str)` — Parse JSON
- `JSONStringify(obj)` — Convert to JSON string

10.12 Data Structures Utilities

- `range(start, end, step)` — Generates numeric range
- `split(str, sep)` — Splits string
- `join(arr, sep)` — Joins array into string
- `substring(str, start, end)` — Extract substring

10.13 Utility Summary

The built-in system in Lopo is designed to provide:

- High-level abstraction for common tasks
- Functional programming support
- Lightweight standard library without external dependencies
- Consistent behavior across runtime environment

11 File Import System

Importing ‘lopo’ modules and external JS libraries.

12 Advanced Concepts

Inheritance, environments, closures, and runtime evaluation model.

13 Appendix A: Syntax Quick Reference

This section provides a quick reference for Lopo syntax. It summarizes the most commonly used language constructs in a compact format.

13.1 Variables

```
define name = value;
```

Examples:

```
define x = 10;
define message = "Hello";
define arr = [1, 2, 3];
```

13.2 Data Types

- String: "text"
- Number: 10, 3.14
- Boolean: true, false
- Array: [1, 2, 3]
- Object: key: value
- Null: null

13.3 Output

```
show value;
```

13.4 Input

```
define name = ask "Enter name";
```

13.5 Operators

- Arithmetic: +, -, *, /
- Comparison: ==, !=, <, >, <=, >=

13.6 If / Else

```
if (condition) {
    show "true";
} else {
    show "false";
}
```

13.7 Loops

13.7.1 For-In Loop

```
for x in arr {  
    show x;  
}
```

13.7.2 For-Let In Loop

```
for let x in arr {  
    show x;  
}
```

13.7.3 While Loop

```
while (condition) {  
    show "loop";  
}
```

13.8 Functions

```
define add = func(a, b) {  
    return a + b;  
};
```

```
show add(2, 3);
```

13.9 Entities

```
entity Person {  
    init(name) {  
        self.name = name;  
    }  
  
    greet() {  
        show self.name;  
    }  
};
```

```
define p = new Person("Alex");  
p.greet();
```

13.10 Inheritance

```
entity Child inherits Parent {  
    init() {  
        self.value = 0;  
    }  
};
```

13.11 Arrays

```
define arr = [1, 2, 3];  
  
arr[0];
```

13.12 Objects

```
define obj = {  
    name: "Lopo",  
    version: 1  
};  
  
obj.name;
```

13.13 Built-in Commands

```
show value;  
ask "question";
```

13.14 Built-in Functions

- len(x)
- lower(x)
- upper(x)
- trim(x)
- toStr(x)
- type(x)
- random(min, max)
- range(a, b)
- keys(obj)
- values(obj)
- map(arr, fn)
- filter(arr, fn)
- reduce(arr, fn)
- push(arr, val)
- pop(arr)
- join(arr, sep)
- split(str, sep)

- `replace(str, a, b)`
- `clone(obj)`
- `deepClone(obj)`
- `JSONParse(str)`
- `JSONStringify(obj)`
- `sleep(ms)`
- `now()`

13.15 File Operations

```
readFile(path);
writeFile(path, content);
appendFile(path, content);
exists(path);
mkdir(path);
deleteFile(path);
```

13.16 Math Utilities

- `floor(x)`
- `ceil(x)`
- `round(x)`
- `abs(x)`
- `pow(a, b)`
- `sqrt(x)`
- `min(...)`
- `max(...)`
- `clamp(value, min, max)`

13.17 String Utilities

- `substring(str, start, end)`
- `padStart(str, len)`
- `padEnd(str, len)`
- `capitalize(str)`
- `reverseStr(str)`
- `trimStart(str)`
- `trimEnd(str)`

13.18 Arrays Utilities

- `unique(arr)`
- `flatten(arr)`
- `indexOf(arr, val)`
- `includesArr(arr, val)`
- `randomChoice(arr)`

13.19 Object Utilities

- `keys(obj)`
- `values(obj)`
- `entries(obj)`
- `merge(obj1, obj2)`
- `invert(obj)`
- `hasOwn(obj, key)`

14 Appendix B: Example Programs

This section contains sample Lopo programs for practice. These examples demonstrate real-world usage patterns of the language.

14.1 Simple Calculator

```
define a = 10;
define b = 5;

define add = func(x, y) {
    return x + y;
};

define sub = func(x, y) {
    return x - y;
};

show add(a, b);
show sub(a, b);
```

14.2 Loop with Condition Filtering

```
define numbers = [1, 2, 3, 4, 5, 6];

for n in numbers {
  if (n % 2 == 0) {
    show n;
  }
}
```

14.3 Object Processing Example

```
define user = {
  name: "Alex",
  age: 22,
  city: "London"
};

for key in user {
  show key + ": " + user[key];
}
```

14.4 Entity Usage Example

```
entity Car {
  init(model, year) {
    self.model = model;
    self.year = year;
  }

  info() {
    show self.model + " (" + self.year + ")";
  }
}

define c = new Car("Tesla", 2024);
c.info();
```

14.5 String Manipulation Example

```
define text = "  lopo language  ";

define cleaned = trim(text);
define upperText = upper(cleaned);

show cleaned;
show upperText;
```

14.6 Array Transformation Example

```
define arr = [1, 2, 3, 4, 5];

define doubled = map(arr, func(x) {
    return x * 2;
});

show doubled;
```

14.7 File Operations Example

```
writeFile("test.txt", "Hello Lopo!");

define content = readFile("test.txt");
show content;
```

14.8 Random Utility Example

```
show random(1, 100);
show randomChoice(["apple", "banana", "mango"]);
```

14.9 Recursive Function Example

```
define factorial = func(n) {
    if (n == 0) {
        return 1;
    }
    return n * factorial(n - 1);
};

show factorial(5);
```